LINKED LISTS

- There are certain drawbacks of using sequential storage to represent arrays.
  i) A fixed amount of storage remains allocated to the array even when the structure is actually using a smaller amount or possibly no storage at all.
  ii) Only fixed amount of storage may be allocated, making overflow a possibility.

- In a sequential representation, the items of an array are implicitly ordered by the sequential order of storage. If arr[x] represents an element of an array, the next element will be arr[x + 1]

- Suppose that the items of an array were explicitly ordered i.e. each item contained within itself the address of the next element. Such an explicit ordering gives rise to a data structure known as a Linear linked list as shown in the figure below.

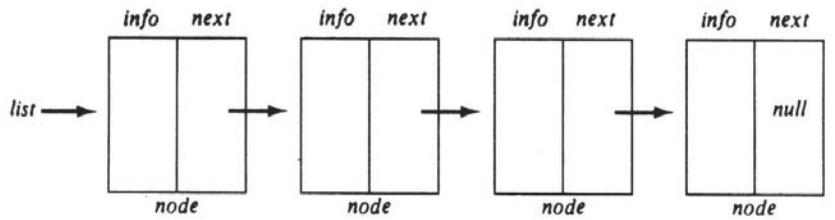

Fig.1: Linear linked list

→ Each item in the list is called a node and it contains two fields.
  i) Information field: It holds the actual element on the list.
  ii) The next address field: It contains the address of the next node in the list.
  Such an address, which is used to access a particular node, is known as a **pointer**.

→ The entire linked list is accessed from an external pointer list that points to (contains the address of) the first node in the list.

→ The next address field of the last node in the list contains a special value called null. This is not a valid address and is used to signal the end of a list.

→ The list with no nodes on it is called the empty list or the null list. The value of the external pointer list to such a list is the null pointer. A list can be initialized to the empty list by the operation list = null.

INSERTING AND REMOVING NODES FROM A LIST

Insertion

A list is a dynamic data structure. The number of nodes on a list may vary as elements are inserted and removed. The dynamic nature of a list may be contrasted with the static nature of an array, whose size remains constant.

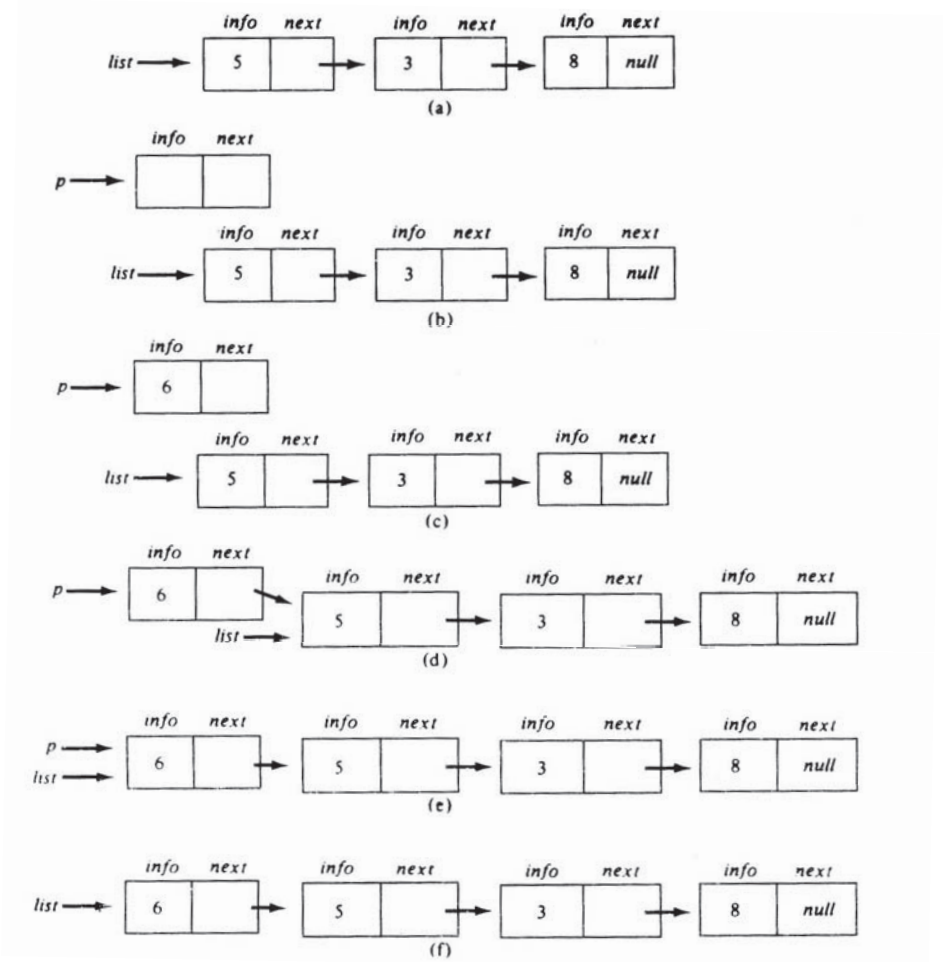

Fig.2: Adding an element to the front of a list

Example :

Suppose that we are given a list of integers as illustrated in figure 2(a) and we have to add the integer 6 to the front of that list i.e. we want the list in figure 2(f).

→ Assume the existence of mechanism for obtaining empty nodes.
The operation p = getnode( );
obtains an empty node and sets the contents of a variable named p to the address of that node. The value of p is then a pointer to this newly allocated node.

→ In figure 2(b), we see the list and the new node after performing the getnode operation. The next step is to insert the integer 6 into the info portion of the newly allocated node. This is done by info (p) = 6; (The result is shown in figure 2(c))

→ After setting the info portion of node (p), it is necessary to set the next portion of that node. Since node (p) is to be inserted at the front of the list, the node that follows should be the current first node on the list. Since the variable *list* contains the address of that first node, node (p) can be added to the list by performing the operation next (p) = *list*;
[This operation places the value of list (which is the address of first node on the list) into the next field of node(p)]

→ At this point, p points to the list with the additional item included. However, since list is the external pointer to the desired list, its value must be modified to the address of the new first node of the list. This can be done by performing the operation *list* = p; which changes the value of *list* to the value of p.
figure 2(e) illustrates the result of this operation.

→ Generalized Algorithm

```
    p   =   getnode( )   [allocate memory for new node]
info(p) =   x;
next(p) =   list ;
 list   =   p;
```

Deletion

The following figure shows the process of removing the first node of a nonempty list and storing the value of its info field into a variable x. The initial step and final step are shown in figure 3(a) and 3(f) respectively.

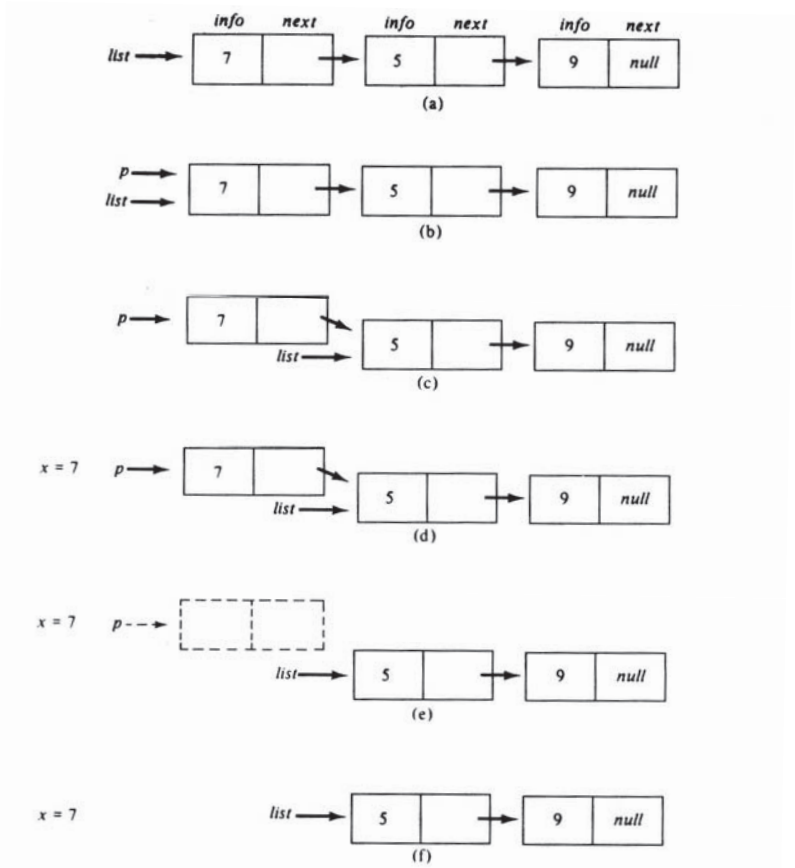

Fig.3: Removing a node from the front of a list

(Note: This process is exactly opposite to the process of adding a node)

→ To obtain figure 3(d) from figure 3(a) the following operations are performed.

p = *list*;
list = next(p);
x = info(p);

→ In figure 3(d), p still points to the node that was formally first on the list. That node is currently useless because it is no longer on the list and its information has been stored in x.
(Note : The node is not considered to be on the list despite the fact that next(p) points to a node on the list, since there is no way to reach node(p) from the external pointer *list*)

→ During the process of removing the first node from the list, the variable p is used as an auxillary variable. The starting and ending configurations of the list make no reference to p. But once the value of p is changed there is no way to access the node at all, since neither an external pointer nor a next field contains its address. Therefore the node is currently useless and cannot be reused.

- The getnode creates a new node, whereas freenode destroys a node. Figure 3(e) and 3(f) depicts a node being freed.

STACK

A Stack is an ordered collection of items into which new items may be inserted or deleted only at one end, called the top of the stack.

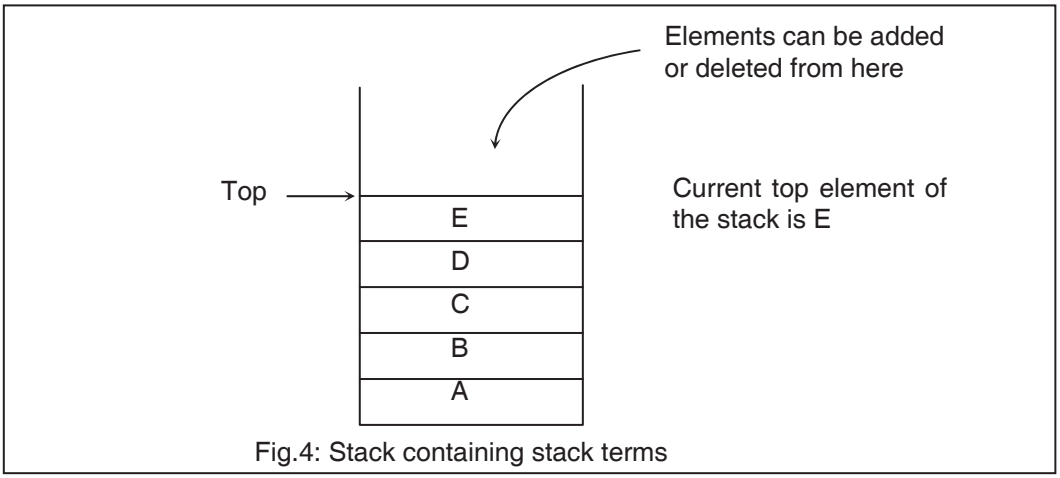

Fig.4: Stack containing stack terms

- A stack is different from an array as it has the provision for the insertion and deletions of items, thus a stack is a dynamic, constantly changing object.

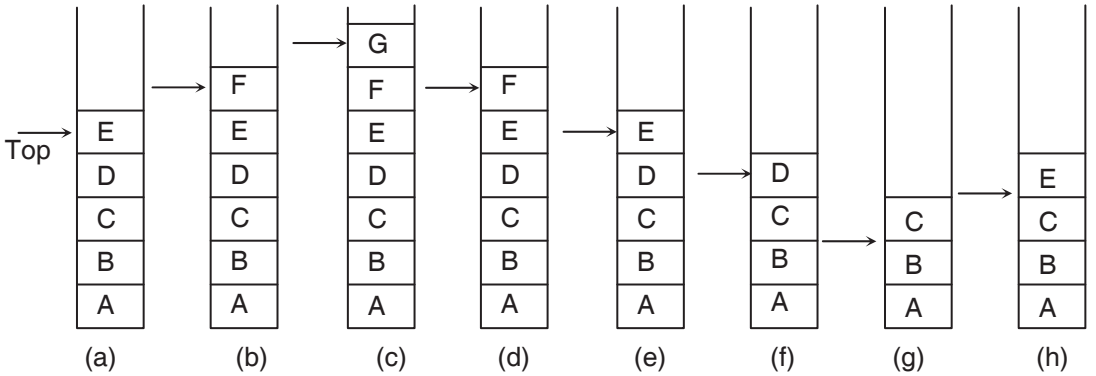

Fig.5: Motion picture of a stack

- The last element inserted into a stack is the first element deleted. Thus stack is called a last–in, First–out (or LIFO) list.
- One cannot distinguish between frame (a) and frame (e) by looking at the stack's state at the two instances. No record is kept on the stack of the fact that two items had been pushed and popped in the meantime.
- The true picture of a stack is given by a view from the top looking down, rather than from a side looking in. Thus there is no perceptible difference between frame (e) and (d) One can compare the stacks at the two frames by removing all the elements on both stacks and compare them individually.

OPERATIONS ON STACK

When an item is added to a stack, it is pushed onto the stack.
When an item is removed, it is popped from the stack.

- Given a stack s and an item i, performing the operation push (s, i) adds item i to the top of stack s.
- The operation

  i = pop(s);

  removes the element at the top of s and assigns its value to i.

As a push operation adds elements onto a stack, a stack is sometimes called a pushdown list.

- There is no upper limit on the number of items that may be kept in a stack, though a pop operation cannot be applied to the empty stack as such a stack has no elements to pop.

- The operation empty(s) determines whether stack s is empty or not. If the stack is empty; empty(s) returns the value TRUE, otherwise it will return the value FALSE.
- The operation stacktop(s) returns the top element of stack s. The stacktop(s) operation can be decomposed into a pop and a push. It is also called as peek operation.

  i = stacktop(s);
  is equivalent to
  i = pop(s);
  push (s, i);

  stacktop is not defined for an empty stack.

The result of an illegal attempt to pop or access an item from an empty stack is called underflow.

Operation	Function
Push(s, i)	Adds item i on the top of stack s
pop(s)	Removes the top element of stack s
empty(s)	Determines whether or not stack s is empty
stacktop(s)	Returns the top element of stack s without popping it

APPLICATIONS OF STACKS

The place where stacks are frequently used is in evaluation of arithmetic expression. An arithmetic expression consists of operands and operators. The operands can be numeric values or numeric variables. The operators used in an arithmetic expression represent the operations like addition, subtraction, multiplication, division and exponentiation.

If the operator is situated in between the operands, the representation is called infix.
If the operator precedes the operands, the representation is called prefix.
If the operator follows the operands, the representation is called postfix.

To fix the order of evaluation of an expression each language assigns to each operator a priority. Even after assigning priorities how can a compiler accept an expression and produce correct code?

A polish mathematician Jan Lukasiewicz suggested a notation called **Polish** notation, which gives two alternatives to represent an arithmetic expression. The notations are **prefix** and **postfix** notations. The fundamental property of **Polish** notation is that the order in which the operation are to be performed is completely determined by the positions of the operators and operands in the expression. Hence, parentheses are not required while writing expressions in **Polish** notation.

While writing an arithmetic expression, the operator symbol is usually placed between two operands. For example,

A + B * C
A * B – C
A + B / C – D
A $ B + C

This way of representing arithmetic expressions is called infix notation. While evaluating an infix expression usually the following operator precedence is used:

- Highest priority: Exponentiation ($)
- Next highest priority: Multiplication (*) and Division (/)
- Lowest priority: Addition (+) and Subtraction (–)

If we wish to override these priorities we can do so by using a pair of parentheses as shown below.

(A + B) *C
A* (B – C)
(A + B) / (C – D)

The expressions within a pair of parentheses are always evaluated earlier than other operations.

In prefix notation the operator comes before the operands. For example, consider an arithmetic expression expressed in infix notation as shown below:

A + B

This expression in prefix form would be represented as follows:

+AB

The same expression in postfix form would be represented as follows:

AB+

In postfix notation, the operator follows the two operands.

The prefix and postfix expressions have three features:

- The operands maintain the same order as in the equivalent infix expression.
- Parentheses are not needed to designate the expressions unambiguously.
- While evaluating the expression the priority of the operators is irrelevant.

- Polish / Prefix notation (Infix to Prefix)

  (A + B) ∗ C = [+AB] ∗ C = * + ABC
  A + (B ∗ C) = A + [∗ BC] = +A ∗ BC
  (A + B) / (C – D) = [+AB] / [–CD] = /+AB – CD

- Reverse polish / postfix Notation (Infix to Postfix)

  A $ B ∗ C – D + E / F / (G + H) = AB $ C ∗ D – EF / GH + /+
  ((A + B) ∗ C – (D – E)) $ (F + G) = AB + C ∗ DE – – FG + $
  A – B / (C ∗ D $ E) = ABCDE $ ∗ / –

- The computer evaluates an arithmetic expression written in infix notation in two steps: first it converts the expression to postfix notation and then it evaluates the postfix expression.

Evaluation of a Postfix Expression

Let P be an arithmetic expression written in postfix notation. The following algorithm evaluates P using a STACK to hold operands.

Algorithm:

1. Add a right parenthesis ")" at the end of P.
2. Scan P from left to right and repeat steps 3 and 4 for each element of P until the ")" is encountered.
3. If an operand is encountered, push it on STACK.
4. If an operator is encountered, then
   a) Remove the two top elements of STACK, where X is the top element and Y is the next–to–top element.
   b) Evaluate Y operator X.
   c) Place the result of (b) back on STACK.
   [End of if structure]
   [End of 2 loop]
5. Set VALUE equal to the top element on STACK.
6. Exit.

Example of above concept:
Consider the following arithmetic expression P written in postfix notation.

P: 4, 5, 2, +, *, 16, 4, /, –

Below are the contents of STACK as each element of P is scanned.

Symbol Scanned	STACK
4	4
5	4, 5
2	4, 5, 2
+	4, 7
*	28
16	28, 16
4	28, 16, 4
/	28, 4
–	24
)	

The final number in STACK is 24, which is assigned to VALUE when ")" is scanned and thus is the final value of P.

Infix to Postfix Conversion

Let Q be an arithmetic expression written in infix notation Q, besides containing operands and operators also contain left and right parentheses.
The following algorithm is used to transform the infix expression Q to its equivalent postfix expression P. It uses a stack to temporarily hold operators and left parentheses. The postfix expression P will be constructed from left to right using the operands from Q and the operators which are removed from STACK. To begin, push a left parenthesis onto STACK and add a right parenthesis at the end of Q. The algorithm is completed when STACK is empty.

Algorithm:

1. Push "(" onto STACK, and add ")" to the end of Q.
2. Scan Q from left to right and repeat steps 3 to 6 for each element of Q until the STACK is empty.
3. If an operand is encountered, add it to P.
4. If a left parenthesis is encountered push it onto STACK.
5. If an operator ⊗ is encountered, then:
   a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) which has the same precedence as or higher precedence than ⊗.
   b) Add ⊗ to STACK
   [End of If structure]
6. If a right parenthesis is encountered, then:
   a) Repeatedly pop from STACK and add to P each operator (on the top of STACK) until a left parenthesis is encountered.
   b) Remove the left parenthesis. [Do not add the left parenthesis to P]
   [End of If structure]
   [End of step 2 loop]
7. Exit

Example of above concept

Consider the following arithmetic infix expression Q:

Q: A + (B ∗ C – (D / E ↑ F) ∗ G) ∗ H

Convert the above infix expression into its equivalent postfix expression P.

Symbol Scanned	STACK	Expression P
A	(	A
+	(+	A
(	(+ (	A
B	(+ (	A B
∗	(+ (∗	A B
C	(+ (∗	A B C
–	(+ (–	A B C ∗
(	(+ (– (	A B C ∗
D	(+ (– (	A B C ∗ D
/	(+ (– (/	A B C ∗ D
E	(+ (– (/	A B C ∗ D E
↑	(+ (– (/ ↑	A B C ∗ D E
F	(+ (– (/ ↑	A B C ∗ D E F
)	(+ (–	A B C ∗ D E F ↑ /
∗	(+ (– ∗	A B C ∗ D E F ↑ /
G	(+ (– ∗	A B C ∗ D E F ↑ / G
)	(+	A B C ∗ D E F ↑ / G ∗ –
∗	(+ ∗	A B C ∗ D E F ↑ / G ∗ –
H	(+ ∗	A B C ∗ D E F ↑ / G ∗ – H
)		A B C ∗ D E F ↑ / G ∗ – H ∗ +

After the last step, the STACK is empty and the postfix equivalent of Q is

P: A B C ∗ D E F ↑ / G ∗ – H ∗ +